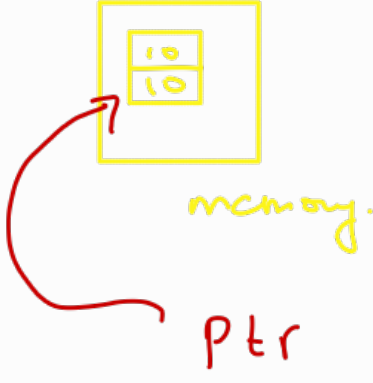
```
class Rectangle
{
    float Len;
    float width;

    Public:
    Rectangle (float length , float width)
    {

    }
    void print ()
    {
        cout<< "length = " << length <<endl;
        cout<< "width = " << width <<endl;
    }
};
```

```
int main ()
{
    Rectangle R1 (10, 10)

    Rectangle *Ptr;
    Ptr = &R1;
}
```



Pto points to the object R1.

To Access members of this object through pointers

use arrow

```
Pto -> print();
```

(Creating a dynamic object)

↳ objects created while the program is running

Fraction *fp1, *fp2, *fp3; → pointers that can store address of objects.

fp1 = new Fraction; → creates one object of class fraction dynamically

→ calls default constructor

→ stores the address in fp1.

(Dot operator vs arrow operator)

↓

used in normal object

obj.a

obj.show()

→ used when we have a pointer to a object

fp1→a

fp1→show()

Same as

*fp1.show()

```
class box {
    int len;
  public
   box() {
       len = 0;
    }
   box (int l)
    {
      len = l;
    }
  void show()
   { cout << "length" << len;
   }
};
```

box *bp;

bp = new box → creates 1 box object and default cons runs

bp → show();

delete bp;

bp = new box (10); → creates 1 box object and parameterized constructor runs

bp → show()

delete bp;

bp = new box [2] → creates two separate box object

bp → show() → will show only the first object

bp[i].show()

To Access members for a Single dynamic object use arrow

fp1 → show();

Dynamically allocated objects

first[3].show();

(Pointer to a data member)

```
class A
{   public:
      int a;
      void take()
      {   cout<<a;
      }
};
```

Datatype classname :: *ptrname

ptrname = &classname :: datamember;

↑ of member

main

A obj1;

int A :: *ptr

ptr = &A :: a;

↓ address

A obj1;

A *ptr = &obj1; → ptr to object

int A :: *p1

p1 = &A :: a] ptr to member

Accessing value

obj1.*ptr = 10;

↓ go to obj1 and access the member ptr points to.

Same

Access obj1.a

ptr → *p1 = 20;

go to obj that ptr points to (obj 1)

*p1 → go to member that p1 points to.

and give me the value

Box *bp ;

bp = new box;

bp → show() pointer to object

---

praxtice.cpp | 24L-0682Q1.cpp | 24L-0682Q1.cpp

ACTICE-1 | (Global Scope) | main()

```cpp
#include<iostream>
using namespace std;

class box{
public:
    int length;
public:
    box(){
        length = 0;
    }
    box(int l)
    {
        length = l;
    }
    void show()
    {
        cout << "Length " << length << endl;
    }
};

int main()
{
    box obj1(10);  //created a object
    box* ptr;    //created a pointer that will point to a object
    ptr = &obj1;
    ptr->show();

    //now using a pointer to point to direct member
    int box ::* ptr1 = &box::length;

    ptr->*ptr1 = 20;
    cout << ptr->*ptr1;  → obj1.ptr1

    obj1.*ptr1 = 10;
    obj1.show();
```

Pointer to Member Functions of class:

Syntax

↗ Function

return type (class name :: * ptr name) (argument type) =

& class-name :: function-name;

```
Class data {
  public:
    int f (float a) {
        return 1;
    }
};
```

int (data :: *ptr) (float)

= &data :: f;

address of member function

ptr is a pointer to a member function of class data.

```
Data obj1;
cout << (d.*ptr)(1);
```

→ calling the function via pointer.

```
Data *ptr;
ptr1 = &obj1;
cout << (ptr1->*ptr)(1);
```

> Another way

(Inline function)

```
int square (int x)
{
    return x * x;
}

int main() {
    cout << square (5);
    return 0;
}
```

Takes time to execute

```
inline int square (int x)
{
    return x * x;
}

int main() {
    cout << square (5);
    return 0;
}
```

Faster as compiler lines the code directly

Situation where Inlining is offered

⇒ Function has loop, static variable, Recursive function

↓

function calls itself

operator overloading obj1 + obj2 (x) cannot do this.

Syntax

Return type operator sign (parameter list)

{

Body of function

}

Void operator ++ () {

}

e.g